

PROJECT REPORT

TITLE: Old Photograph Restoration Using Image Enhancement and Blur Reduction

Submitted by: Anaha Shaji

Reg no: 24ubc107

Department: BCA

Marian College, Kuttikkanam

GitHub Repository:

<https://github.com/AnahaShaji/Old-Photograph-Restoration-DIP>

1. Problem Statement

Old photographs often suffer from blur, loss of sharpness, poor contrast, and other forms of image degradation due to aging and storage conditions. These degradations reduce the visual quality of photographs and make important details difficult to recognize.

The problem addressed in this project is to improve the quality of degraded vintage photographs using Digital Image Processing techniques. Different image enhancement and restoration methods are applied to reduce the effect of blur and improve image sharpness and visual quality.

The project also compares the performance of different techniques using MSE, PSNR, and SSIM to identify the most effective method.

Problem Relevance

Restoring old photographs can be useful for personal photo preservation, historical archives, museums, and digital collections. Image processing provides a simple way to improve the appearance and readability of degraded photographs.

2. Objectives

The main objectives of the project are:

1. To improve the visual quality of degraded vintage photographs.
2. To reduce the effect of blur using image processing techniques.
3. To apply different restoration and enhancement techniques.
4. To compare the performance of different restoration methods.
5. To evaluate the results using MSE, PSNR, and SSIM.
6. To identify the most effective technique for the selected photographs.

3. Dataset / Image Description

The project uses the **Vintage Photo Restoration Clean and Damaged Image Dataset** obtained from Kaggle.

The dataset contains vintage photographs with different levels of image quality and degradation.

Item	Details
Dataset	Vintage Photo Restoration Clean and Damaged Image
Source	Kaggle
Total Images	200
Clean Images	146
Damaged Images	54
Image Format	JPG
Input	Vintage photograph
Target	Restored/enhanced photograph

The clean images were used as reference images for the restoration experiments. 10 clean images were selected for the main experiment.

The images were loaded using OpenCV. Gaussian blur was applied to the selected clean images to simulate image degradation. The images were converted to grayscale where required for filtering and quantitative evaluation.

The real damaged images were mainly used for qualitative observation, as corresponding clean ground-truth images were not available.

Dataset Limitations

The dataset contains a limited number of real damaged photographs without corresponding clean ground-truth images. Therefore, quantitative evaluation was performed using clean images with simulated Gaussian blur.

4. Methodology

The project follows a simple image restoration and enhancement process. Clean vintage photographs are first selected and degraded using Gaussian blur. Different image processing techniques are then applied to improve their quality.

Project Workflow

Original Image → Gaussian Blur → Restoration / Enhancement → Result
Comparison → Evaluation

Steps

1. Image Selection: 10 clean vintage photographs were selected from the dataset.
2. Preprocessing: The images were loaded using OpenCV and converted to grayscale where required.
3. Blur Simulation: Gaussian blur with a 15×15 kernel was applied to simulate image degradation.
4. Restoration and Enhancement: Median Filtering, Laplacian Sharpening, Unsharp Masking, and High-Boost Filtering were applied.
5. Contrast Enhancement: Histogram Equalization and CLAHE were tested to improve image contrast.
6. Combined Restoration: Median filtering, CLAHE, and sharpening were combined to create a simple restoration approach.
7. Evaluation: MSE, PSNR, and SSIM were calculated by comparing the processed images with the original clean images.
8. Comparison: The average performance of the methods across 10 images was compared to identify the best-performing technique.

5. Implementation

The project was implemented using Python and commonly used Digital Image Processing libraries.

Tools and Libraries

- Python – Used for implementing the project.
- OpenCV – Used for image processing, filtering, and enhancement.
- NumPy – Used for numerical image operations.
- Matplotlib – Used for image visualization and result graphs.
- Pandas – Used to organize evaluation results.
- Scikit-image – Used for calculating MSE, PSNR, and SSIM.
- KaggleHub – Used to download the dataset.

DIP Techniques Used

The following techniques were implemented:

- Gaussian Blur
- Median Filtering
- Laplacian Sharpening
- Unsharp Masking
- High-Boost Filtering
- Histogram Equalization
- CLAHE
- Combined Image Restoration

Evaluation Metrics

MSE: Measures the pixel-level difference between the original and processed image. Lower values indicate less error.

PSNR: Measures the quality of the processed image compared with the original. Higher values generally indicate better similarity.

SSIM: Measures structural similarity between the original and processed image. A value closer to 1 indicates greater structural similarity.

6. Results

The restoration techniques were evaluated on 10 selected vintage photographs.

Method	MSE	PSNR (dB)	SSIM
Blurred	185.40	26.95	0.7732
Median Filtering	197.48	26.67	0.7605
Laplacian Sharpening	159.35	27.62	0.7840
Unsharp Masking	154.32	27.83	0.7990
High-Boost Filtering	2651.35	14.13	0.7098
Combined Restoration	654.74	20.22	0.7415

Result Analysis

The results show that Unsharp Masking achieved the best overall performance among the tested techniques.

It obtained:

- Lowest MSE: 154.32
- Highest PSNR: 27.83 dB
- Highest SSIM: 0.7990

Laplacian Sharpening also performed well, with an average PSNR of 27.62 dB and SSIM of 0.7840.

Median Filtering showed limited improvement because it mainly performs smoothing and noise reduction rather than directly reversing blur.

High-Boost Filtering produced excessive enhancement and resulted in poor quantitative performance.

The Combined Restoration method also showed lower quantitative performance because combining multiple enhancement operations introduced additional pixel-level changes.

Overall, Unsharp Masking was the most effective technique for the selected images and experimental setup.

Visual Comparison

The visual results showed that:

- Blurred image: Noticeable loss of sharpness.
- Median Filtering: Produced a smoother image but limited sharpening.
- Laplacian Sharpening: Improved edge sharpness.
- Unsharp Masking: Produced a balanced and sharper result.
- High-Boost Filtering: Produced excessive enhancement.
- Combined Restoration: Improved some visual characteristics but introduced noticeable pixel-level changes.

The visual comparison and quantitative evaluation together indicate that Unsharp Masking provided the best overall result.

7. Screenshots

1. Dataset contents and image count

```
import os
# Display the contents of the dataset folder
print("Dataset contents:")
for item in os.listdir(dataset_path):
    print(item)
```

Dataset contents:
image_classification_results.csv
02_Damaged_Testing_Set
01_Clean_Candidates_GT

2. Sample clean and damaged photographs

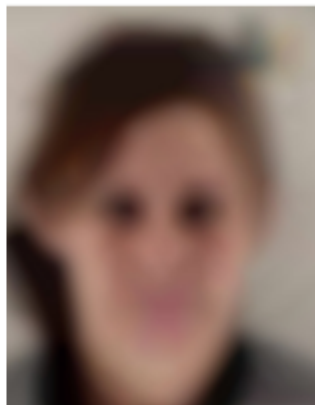
```
import cv2
import matplotlib.pyplot as plt
# Select the first image from each folder
clean_file = clean_images[0]
damaged_file = damaged_images[0]
# Read the images
clean_image = cv2.imread(os.path.join(clean_path, clean_file))
damaged_image = cv2.imread(os.path.join(damaged_path, damaged_file))
# Convert BGR to RGB for displaying with Matplotlib
clean_image = cv2.cvtColor(clean_image, cv2.COLOR_BGR2RGB)
damaged_image = cv2.cvtColor(damaged_image, cv2.COLOR_BGR2RGB)
# Display the images
plt.figure(figsize=(6, 4))
plt.subplot(1, 2, 1)
plt.imshow(clean_image)
plt.title("Clean Reference Image")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(damaged_image)
plt.title("Damaged Image")
plt.axis("off")
plt.show()
```

...

Clean Reference Image



Damaged Image



3. Original and blurred image

```
# Apply Gaussian blur
blurred_image = cv2.GaussianBlur(image_rgb, (15, 15), 0)
# Display the original and blurred images
plt.figure(figsize=(6, 4))
plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Clean Image")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(blurred_image)
plt.title("Blurred Image")
plt.axis("off")
plt.show()
```

Original Clean Image



Blurred Image



4. Median Filtering result

```
# Apply median filtering
median_restored = cv2.medianBlur(blurred_image, 5)
# Display the blurred and median-filtered images
plt.figure(figsize=(6, 4))
plt.subplot(1, 2, 1)
plt.imshow(blurred_image)
plt.title("Blurred Image")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(median_restored)
plt.title("Median Filtered Image")
plt.axis("off")
plt.show()
```

Blurred Image



Median Filtered Image



5. Laplacian Sharpening result

```
▶ # Convert the blurred image to grayscale
blurred_gray = cv2.cvtColor(blurred_image, cv2.COLOR_RGB2GRAY)
# Apply Laplacian operator
laplacian = cv2.Laplacian(blurred_gray, cv2.CV_64F)
# Create sharpened image
laplacian_sharpened = cv2.convertScaleAbs(blurred_gray + laplacian)
# Display the results
plt.figure(figsize=(6, 4))
plt.subplot(1, 2, 1)
plt.imshow(blurred_gray, cmap="gray")
plt.title("Blurred Image")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(laplacian_sharpened, cmap="gray")
plt.title("Laplacian Sharpened Image")
plt.axis("off")
plt.show()
```

...

Blurred Image



Laplacian Sharpened Image



6. Unsharp Masking result

```
▶ # Convert the blurred image to grayscale
blurred_gray = cv2.cvtColor(blurred_image, cv2.COLOR_RGB2GRAY)
# Create a smoothed version
smooth = cv2.GaussianBlur(blurred_gray, (9, 9), 0)
# Apply unsharp masking
unsharp = cv2.addWeighted(blurred_gray, 2.0, smooth, -1.0, 0)
# Display the results
plt.figure(figsize=(6, 4))
plt.subplot(1, 2, 1)
plt.imshow(blurred_gray, cmap="gray")
plt.title("Blurred Image")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(unsharp, cmap="gray")
plt.title("Unsharp Masking")
plt.axis("off")
plt.show()
```

...

Blurred Image



Unsharp Masking



7. High-Boost Filtering result

```
▶ # Create a smoothed version of the blurred image
smooth = cv2.GaussianBlur(blurred_gray, (9, 9), 0)
# Calculate the detail mask
detail = cv2.subtract(blurred_gray, smooth)
# Apply high-boost filtering
high_boost = cv2.addWeighted(blurred_gray, 1.5, detail, 2.0, 0)
# Display the results
plt.figure(figsize=(6, 4))
plt.subplot(1, 2, 1)
plt.imshow(blurred_gray, cmap="gray")
plt.title("Blurred Image")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(high_boost, cmap="gray")
plt.title("High-Boost Filtered Image")
plt.axis("off")
plt.show()
```



8. Histogram Equalization and CLAHE results

```
# Apply Histogram Equalization
hist_equalized = cv2.equalizeHist(input_blurred_gray)
# Create CLAHE object
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
# Apply CLAHE
clahe_result = clahe.apply(input_blurred_gray)
# Display the results
plt.figure(figsize=(6, 4))
plt.subplot(1, 3, 1)
plt.imshow(input_blurred_gray, cmap="gray")
plt.title("Blurred Image")
plt.axis("off")
plt.subplot(1, 3, 2)
plt.imshow(hist_equalized, cmap="gray")
plt.title("Histogram Equalization")
plt.axis("off")
plt.subplot(1, 3, 3)
plt.imshow(clahe_result, cmap="gray")
plt.title("CLAHE")
plt.axis("off")
plt.tight_layout()
```

Blurred Image



Histogram Equalization

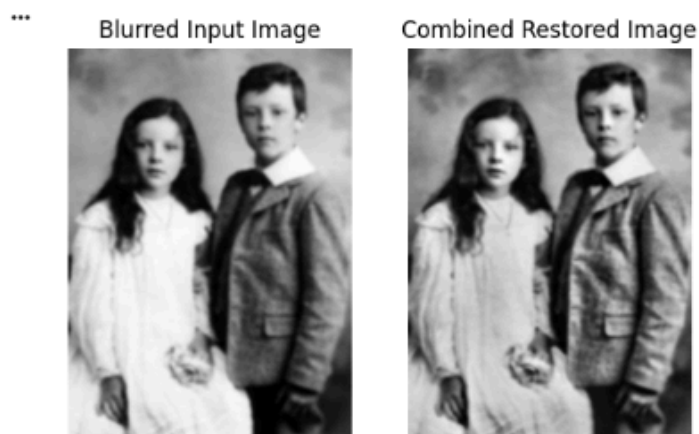


CLAHE



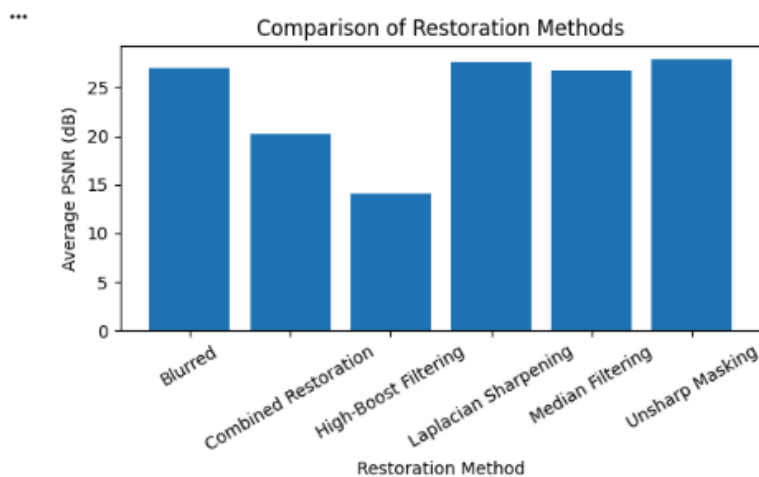
9. Comparison of restoration methods

```
▶ # Step 1: Reduce small noise
combined = cv2.medianBlur(input_blurred_gray, 3)
# Step 2: Improve contrast using CLAHE
combined = clahe.apply(combined)
# Step 3: Enhance edges using unsharp masking
smooth_combined = cv2.GaussianBlur(combined, (9, 9), 0)
combined_restored = cv2.addWeighted(
    combined, 1.5, smooth_combined, -0.5, 0
)
# Display the final restored image
plt.figure(figsize=(6, 4))
plt.subplot(1, 2, 1)
plt.imshow(input_blurred_gray, cmap="gray")
plt.title("Blurred Input Image")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(combined_restored, cmap="gray")
plt.title("Combined Restored Image")
plt.axis("off")
plt.show()
```



10. Average PSNR comparison graph

```
import matplotlib.pyplot as plt
plt.figure(figsize=(6, 4))
plt.bar(
    average_results["Method"],
    average_results["PSNR (dB)"]
)
plt.xlabel("Restoration Method")
plt.ylabel("Average PSNR (dB)")
plt.title("Comparison of Restoration Methods")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```



8. Limitations

The project has the following limitations:

1. Gaussian blur was used to simulate image degradation instead of using paired real damaged and clean images.
2. Traditional image processing techniques cannot fully recover details that are completely lost due to severe blur.
3. The performance of the restoration methods depends on the selected filter parameters.
4. High-Boost Filtering may produce excessive enhancement and unwanted changes in some images.
5. The dataset contains a limited number of real damaged photographs for quantitative comparison.

9. Future Scope

The project can be improved in the future by:

1. Using advanced deblurring techniques to recover more details from severely blurred photographs.
2. Applying automatic parameter selection for different levels of image degradation.
3. Combining noise removal, deblurring, and color restoration techniques.
4. Using deep learning-based image restoration methods for better results.
5. Testing the system on a larger collection of real damaged photographs.
6. Using paired ground-truth images for more accurate quantitative evaluation.
7. Developing the system into a web or mobile application.

10. Conclusion

This project presented a simple image restoration approach for improving the quality of vintage photographs affected by blur and other degradation.

Different spatial filtering and enhancement techniques were implemented and compared using MSE, PSNR, and SSIM. Among the tested methods, Unsharp Masking achieved the best overall performance, with an average MSE of 154.32, PSNR of 27.83 dB, and SSIM of 0.7990 across 10 images.

The results show that simple Digital Image Processing techniques can improve the sharpness and visual quality of degraded photographs. The project also demonstrated that different techniques perform differently depending on the type and severity of image degradation.

Overall, the project successfully demonstrates the application of Digital Image Processing techniques for vintage photograph restoration and enhancement.